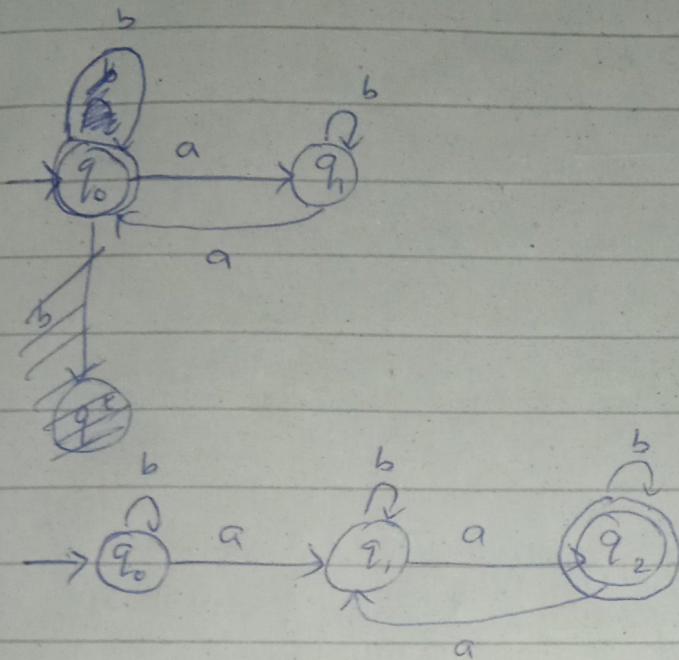
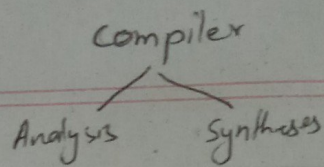


even no. of a's



Model of compilation



Analysis part

- lexical analyze → Linear Analysis / Scanning
- Syntax analyze → ~~Intermediate code~~
- Semantic analyze → Hierarchical Analysis
- Intermediate code → Semantic analysis

lexical analyze The code is divided into separate parts ^{lexeme}
 first phase and while spaces are removed
 int a = 5;

tokens	
int	dt (datatype)
a	identifier
=	assignment operator
5	number
;	
inta	identifier

▲ Symbol table

* only generating the symbol table not checking errors.

* lexems lexeme

* tokens

* Patterns

Syntax analyze ^(Parsing) → syntax tree

Semantic analyze → ~~mean~~ meaningfulness of the program. check if the program is logically correct or not. (tree generated)

Analysis
Frontend

Synthesis
backend

↳ more focused on developing your target language

↳ code optimizer
 ↳ code generation

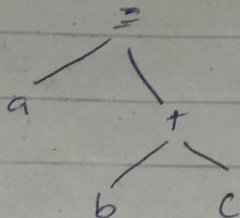
a = b + c

x — x

Position
=
initial
+
rate
*
Go

position
1

$a = b + c$

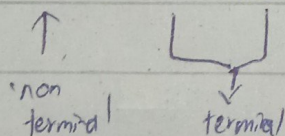


• syntax tree

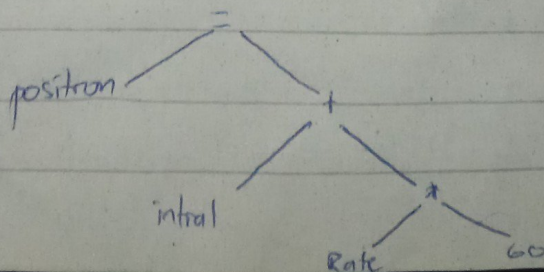
x — x Position: = initial + rate * 60

Position:	identifier
=	assignment operator
initial	identifier
+	Addition Operator
rate	identifier
*	multiplication operator
60	number

$A \rightarrow AaA \mid b$



$$\text{position} = \overbrace{\text{initial}}^{2.1} + \overbrace{\text{rate} * 60}^{2.2}$$



• Intermediate code generation is not the actual code

• Every phase interacts with symbol-table either retrieving or adding something in the table.

• Every phase also interacts with Error handler.

↳ Structure Editor

↳ Pretty printer

↳ Static checkers

↳ Interpreter

initial + rate * 60

~~S → initial A~~

~~A → + B~~

~~B → rate C~~

~~C → * D~~

~~D → 60~~

~~S → a S b | + S | *~~

~~S → a S b A | +~~

~~A → * B~~

~~B → c~~

S → a D b A

A → * B

B → c

D → +

a S b A

a a S b A A x

~~a S b~~

~~a + S b~~

~~a + a S b~~

~~a B~~

a D b A

a + b * B

a + b * c

S → E

E → E + E

E → identifier

Tree

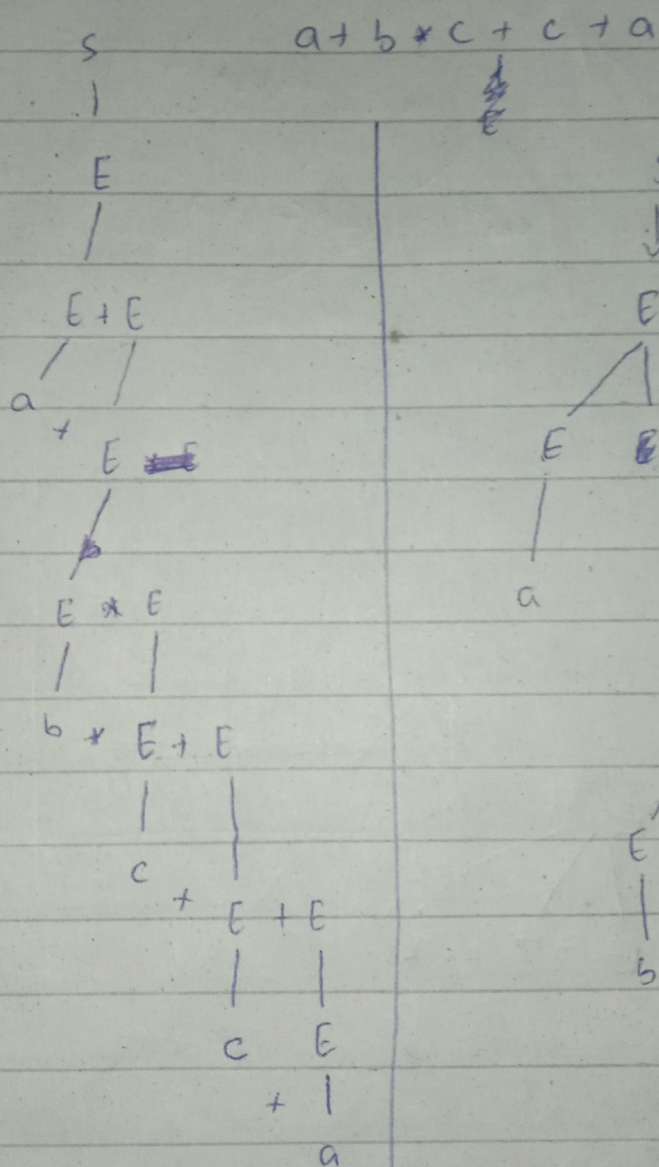
ending on

$$S \rightarrow E$$

$$E \rightarrow E + E \mid E * E$$

$$E \rightarrow \text{identifier } a \mid b$$

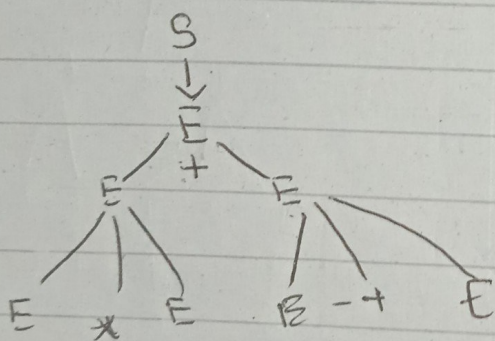
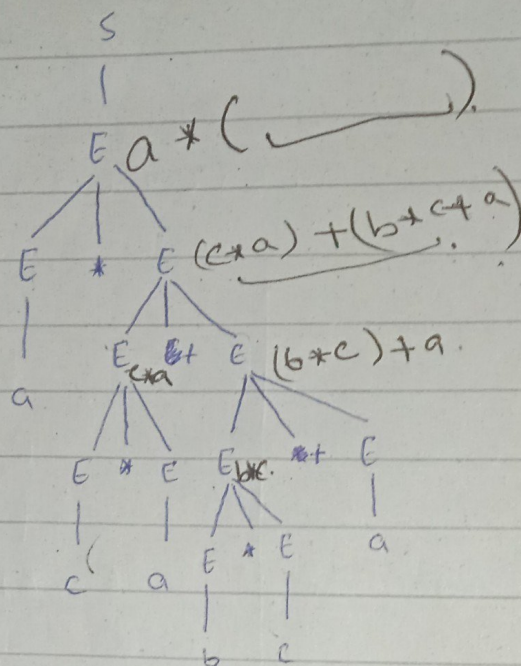
Tree



DBA

3

a * c * a + b * c + a



int a;
int b;
int a, b;

$S \rightarrow EA \mid EA, A$
 $E \rightarrow \text{int} \mid \text{var} \mid \text{string}$
 $A \rightarrow a \mid b \mid d$

~~$S \rightarrow EBAB$~~

~~$E \rightarrow \text{int} \mid \text{var} \mid \text{string}$~~

~~$B \rightarrow ; \mid , \mid +$~~

~~$A \rightarrow$~~

Context free

• set of rules

• we use

English:

$S \rightarrow$

verb

subject

object

A Co

I eat a

A syntax

Context Free Grammar (CFG)

• set of rules that is defining the structure of a language

• we use Terminal and non Terminal

$A \rightarrow a b A A$

↓ ↘
nonterminal terminal
is a replaceable actual symbols of
variable the language

English:

$S \rightarrow$ Subject Verb Object

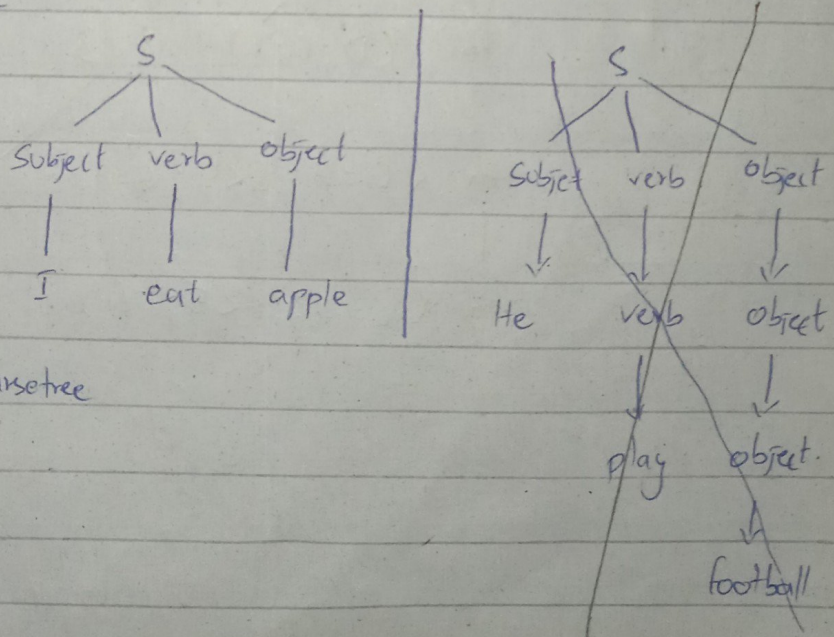
verb $\rightarrow$ eat | play | read

subject $\rightarrow$ I | He | She

object $\rightarrow$ apple | Football | book

▲ Context free Grammar

I eat apple



▲ Syntax / Parse tree

for (i=1; i<5; i++) { }

$S \rightarrow ABC$

$A \rightarrow \text{for}$

$B \rightarrow (D; E; F)$

$D \rightarrow G=H \mid G>H \mid G<H \mid G<=H \mid G>=H \mid$

$E \rightarrow G<H \mid G>H \mid G>=H \mid G<=H \mid$

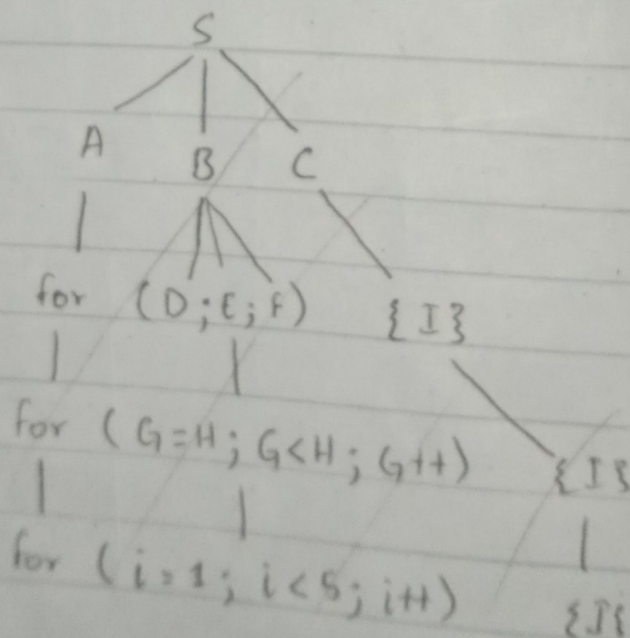
$F \rightarrow G++ \mid G--$

$G \rightarrow a \mid b \mid c \mid d \mid e \mid i$

$H \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid 10 \mid 11 \mid 12$

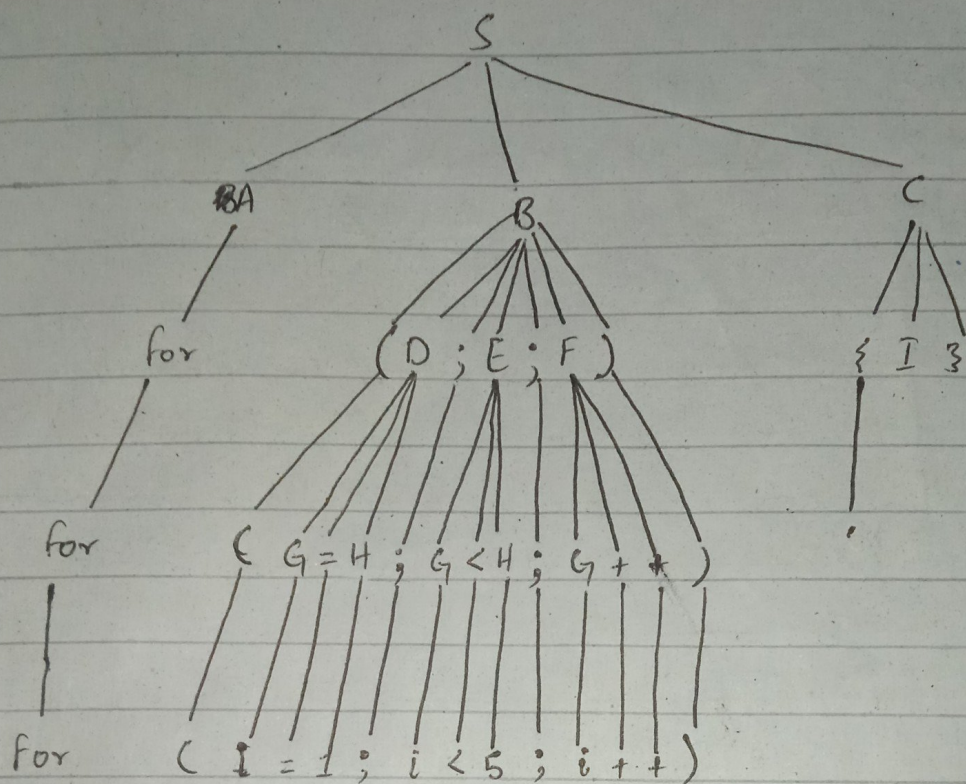
$C \rightarrow \{ I \}$

$I \rightarrow$



for
for

int a, b
sum
sum1



```
int a, b, c
```

Sum

sum123

Regular Grammar :-

letter (letter + number)^{*}

CFG:-

$$S \rightarrow AB$$
$$B \rightarrow AB \mid DB \mid \epsilon$$
$$AC \rightarrow a|b|c|d$$

D → 1 | 2 | 3 | 4 | 5

a
abc
a1

$\langle \text{iden} \rangle \rightarrow \langle \text{letter} \rangle \langle \text{var} \rangle$

$\langle \text{var} \rangle \rightarrow \langle \text{letter} \rangle \langle \text{var} \rangle \mid \langle \text{number} \rangle \langle \text{var} \rangle \mid \epsilon$

$\langle \text{letter} \rangle \rightarrow a|b|\dots|z \mid A|\dots|Z$

$\langle \text{number} \rangle \rightarrow 1|2|3|\dots|9$

int a;

int a, b, c;

$S \rightarrow EAH$

$E \rightarrow \text{int} \mid \text{var} \mid \text{string}$

$A \rightarrow BC$

$C \rightarrow \epsilon \mid ,C \mid BC$

$B \rightarrow DF$

$F \rightarrow DF \mid GF \mid \epsilon$

$D \rightarrow a|\dots|z \mid A|\dots|Z$

$G \rightarrow 1|2|3|\dots|9$

$H \rightarrow ;$

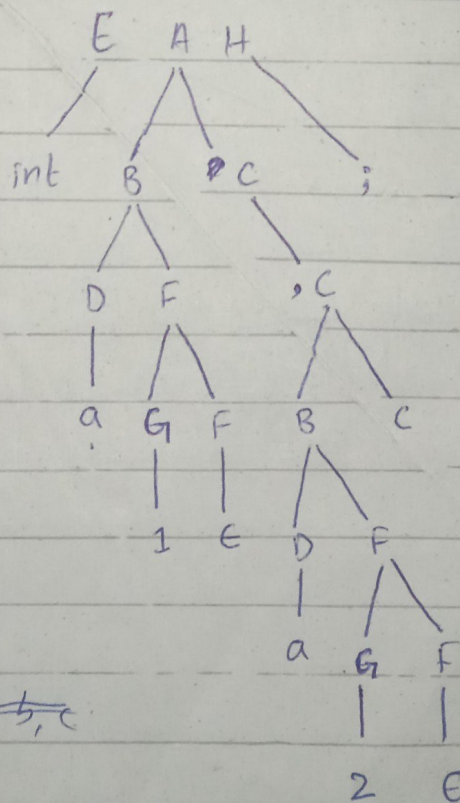
a


```

      E   A H
      |   ^ \
int   BC ;
      |   ^ \
int   DF , C
      ||    |
      ac    BC
           ^ \
           DF E
           ||
          be

```

```
int a1, a2;
```

~~a b c~~

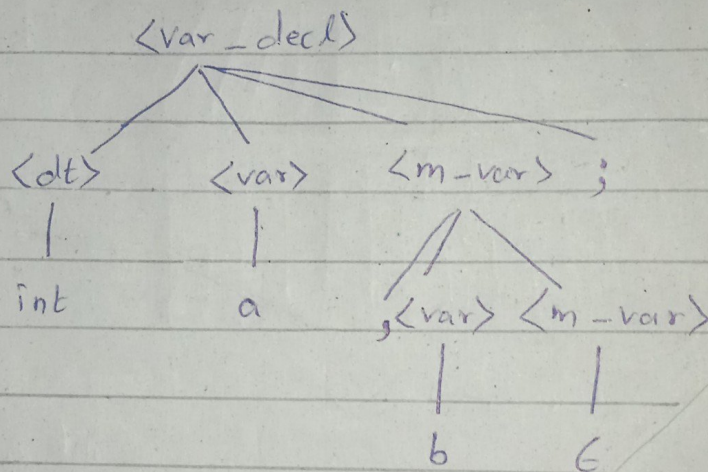
int a, b, c;

$\langle \text{var_decl} \rangle \rightarrow \langle \text{dt} \rangle \langle \text{var} \rangle \langle \text{m-var} \rangle ;$

$\langle \text{m-var} \rangle \rightarrow , \langle \text{var} \rangle \langle \text{m-var} \rangle | \epsilon$

$\langle \text{var} \rangle \rightarrow \langle \text{identifier} \rangle$

$\langle \text{dt} \rangle \rightarrow \text{int} | \text{string} | \text{var}$



int a, b;

→ if

$\langle \text{if} \rangle \rightarrow \text{if} (\langle \text{cond} \rangle) \{ \text{statement} \} |$

$\text{if} (\langle \text{cond} \rangle) \{ \text{statement} \} \text{else } (\langle \text{cond} \rangle) \{ \text{statement} \}$

lexical

① lexen

② Tok

③ Patt

posita

< tok

posi <

inhal < i

rate < id

+ < +

* < *

→ letter

Token

→ gene

→ not

lexical Analyze.

- ① lexeme
- ② Tokens
- ③ Pattern

position = initial + rate * 60

(token, attribute value)

posi < id, 1 >

initial < id, 2 >

rate < id, 3 >

+ < +, >

* < *, >

→ letter (letter + number)*

Token

→ generalized abstract assigned to each lexeme
→ not unique

Transition diagram (NFA)

↳ to generate tokens

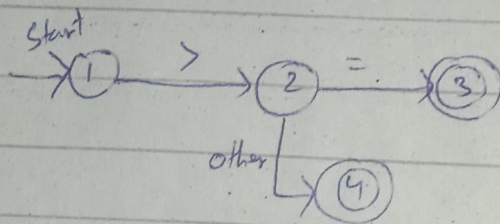
$> \rightarrow (\text{relop}, \text{GT})$

$<$

$\geq (\text{relop}, \text{GTE})$

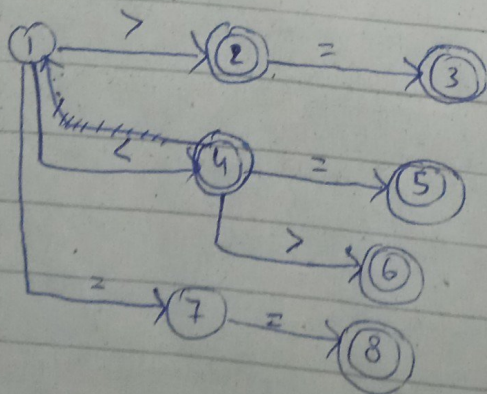
$\leq$

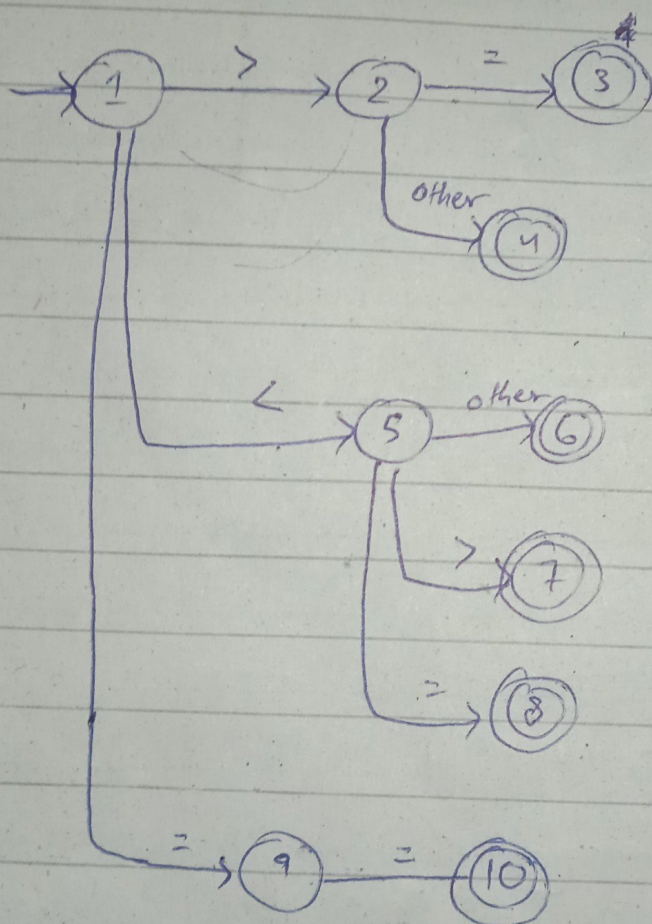
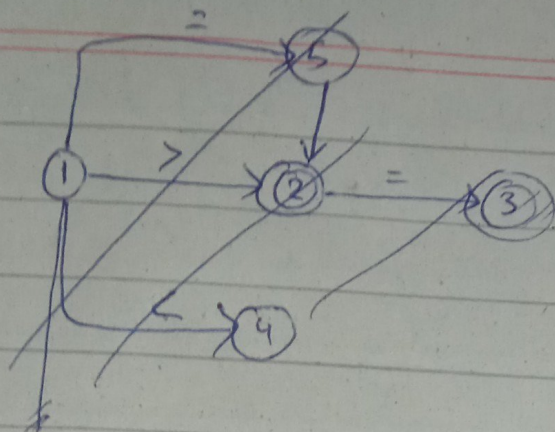
$L \geq 5$



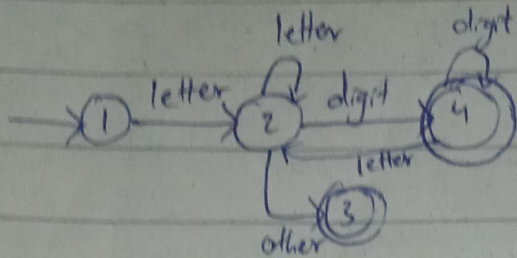
Relational operators

$>$	$\langle \text{relop}, \text{GT} \rangle$
$\geq$	$\langle \text{relop}, \text{GTE} \rangle$
$\leq$	$\langle \text{relop}, \text{LTE} \rangle$
$<$	$\langle \text{relop}, \text{NE} \rangle$
$=$	$\langle \text{relop}, \text{EQ} \rangle$



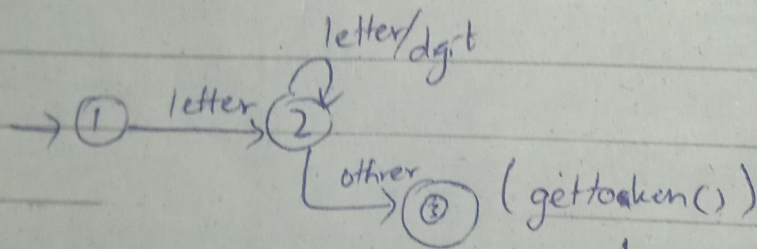
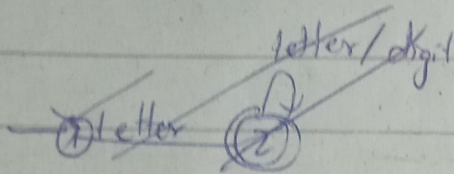


letter (letter + digit)*



letter

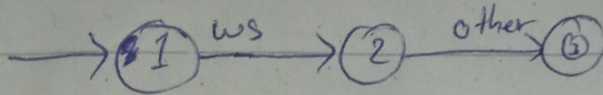
letter a | b



get token() will aggregate
 or separate keyword
 and identifier
 for
 if
 else

whitespace

other



Unsigned Numbers

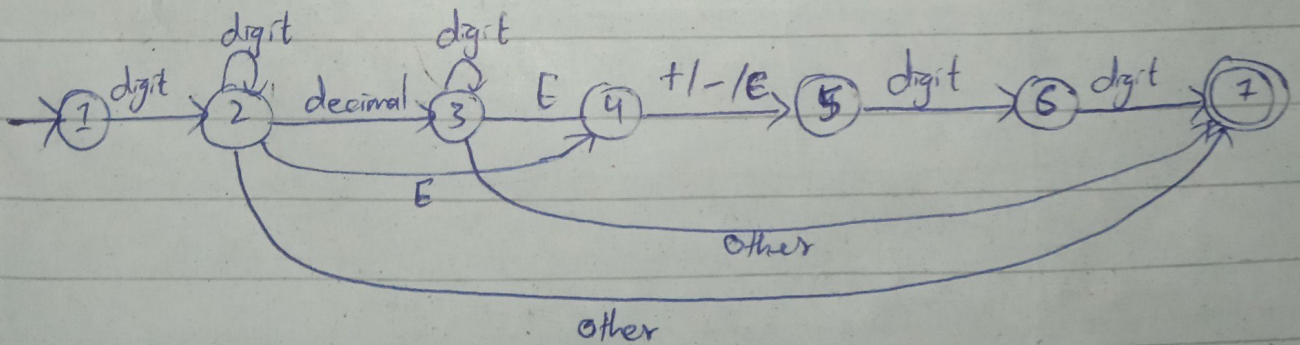
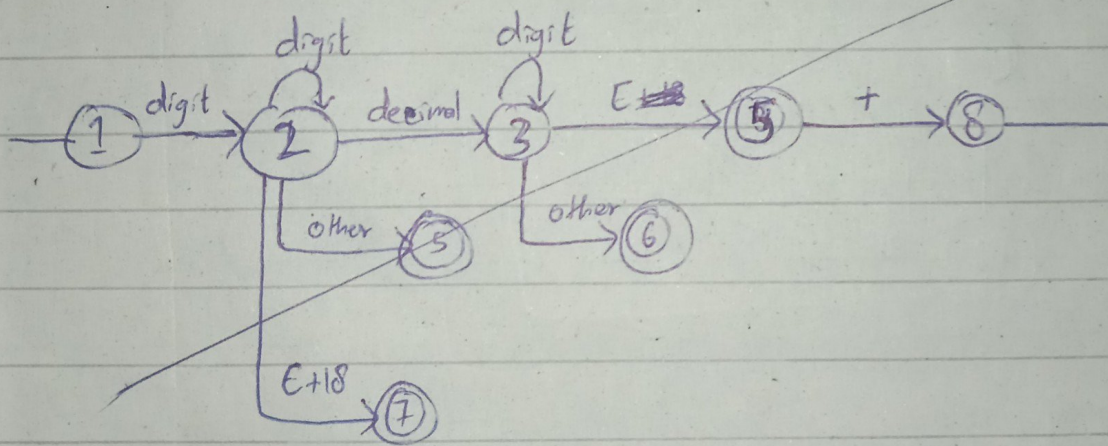
1959500

19.58

19.58E+18

19E+18

other (white space)



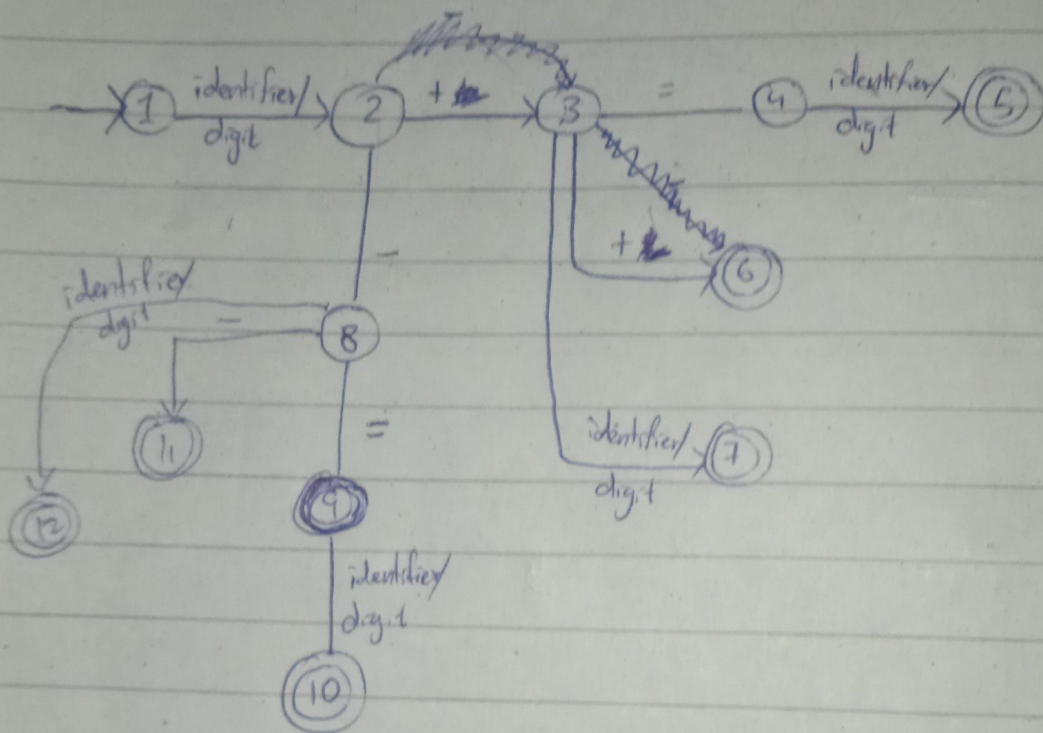
$a + 5$

$a ++$

$5++$

$a + b$

$5+5$

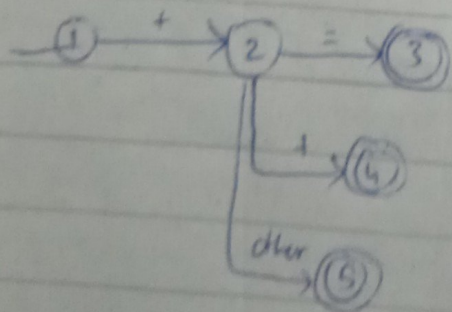


$++$

$+=$

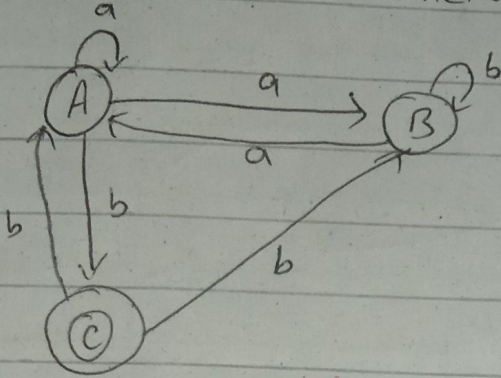
$+$

* Separate transition diagrams
for $+$ and $-$



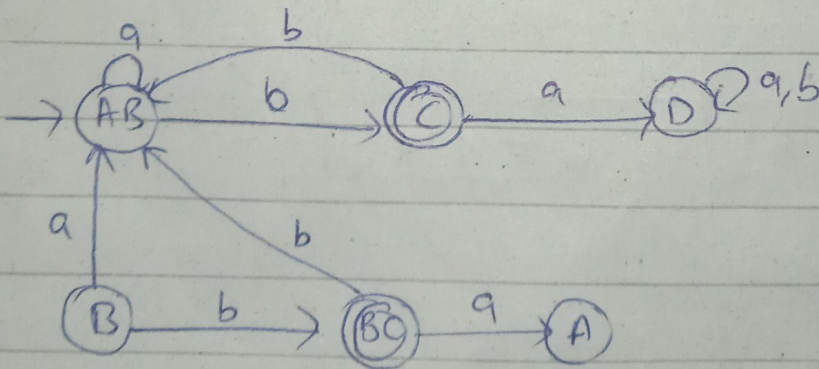
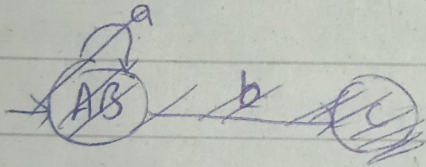
NFA to DFA

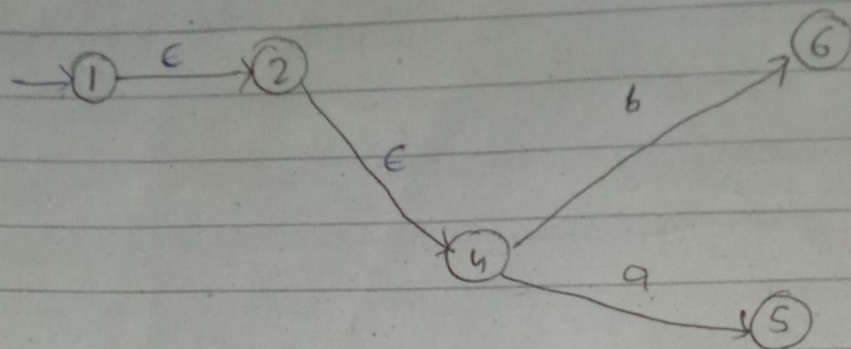
→ subset construction method



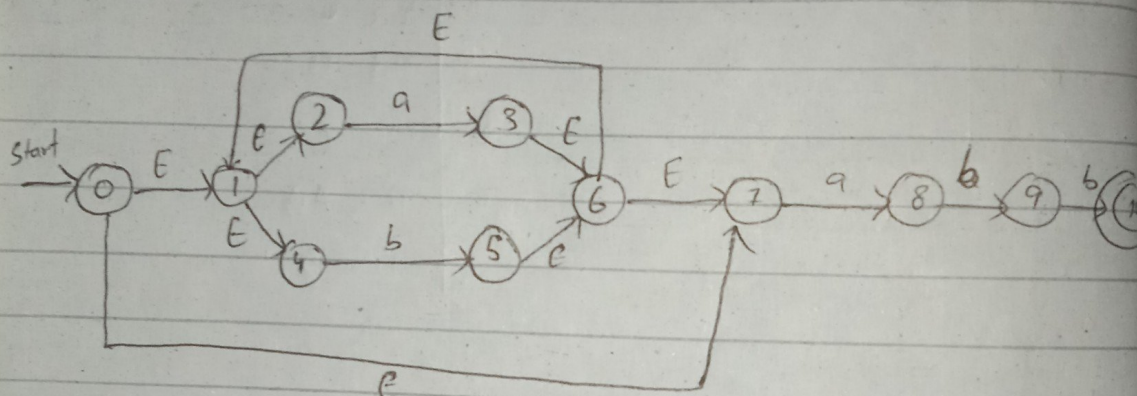
	a	b
A	A, B	C
B	A	B
C	—	A, B

	a	b
→ A	A, B	C
B	A	B
BC	A	AB
C	D	AB
D	D	D



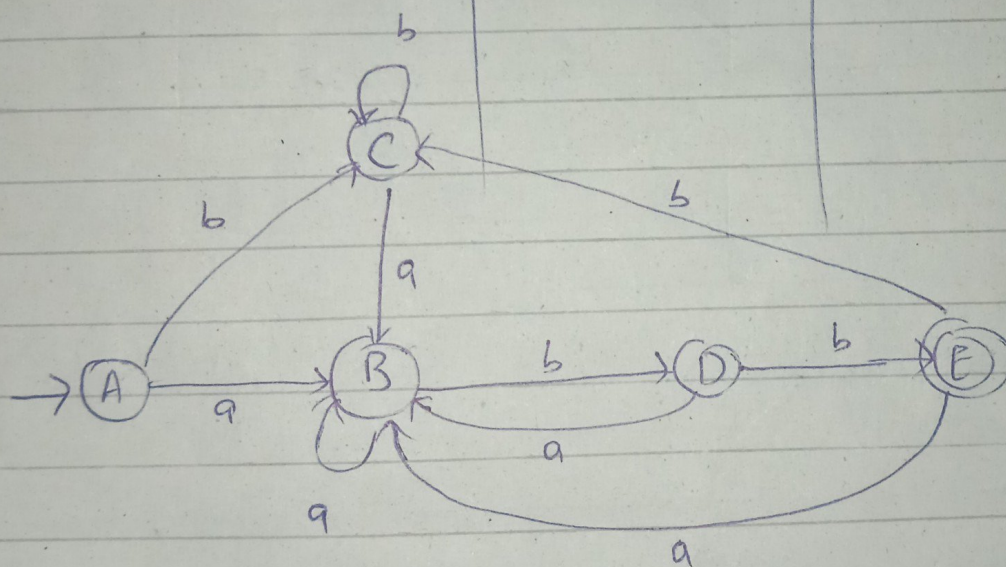


initial states combination $\{1, 2, 4\}$ without reading anything (null)



	a	b
A $\{0, 1, 2, 4, 7\}$	$\{3, 7, 8, 6, 1, 2, 4\}$	$\{5, 6, 7, 1, 4, 2\}$
B $\{1, 2, 3, 4, 7, 8\}$	$\{1, 2, 3, 4, 6, 7, 8\}$	$\{2, 4, 6, 9, 5, 7, 1\}$
D $\{1, 2, 4, 6, 5, 7, 9\}$	$\{3, 6, 1, 4, 2, 7, 8\}$	$\{4, 5, 6, 1, 10, 7, 2\}$
C $\{1, 2, 4, 5, 6, 7\}$	$\{3, 6, 1, 4, 2, 7, 8\}$	$\{5, 6, 7, 4, 2, 4\}$

ϵ	a	b
$\{1, 2, 4, 5, 6, 7, 10\}$	$\{3, 6, 1, 4, 2, 7, 8\}$	$\{5, 6, 7, 1, 2, 4\}$



Syntax analysis should have a CFG to check
the syntax

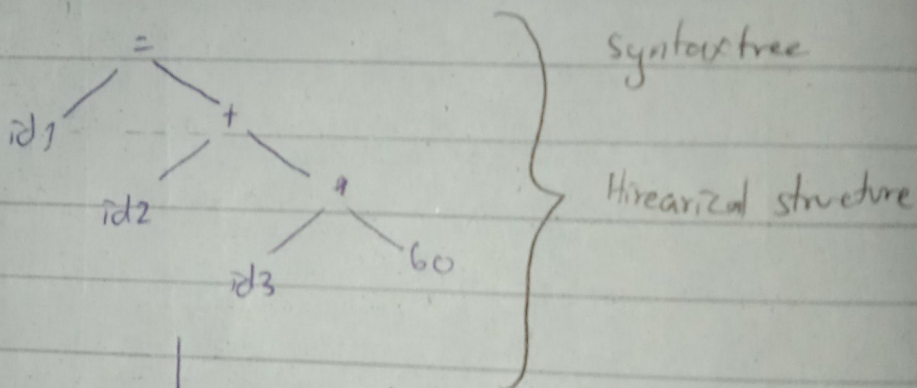
$a = b + c * 60$



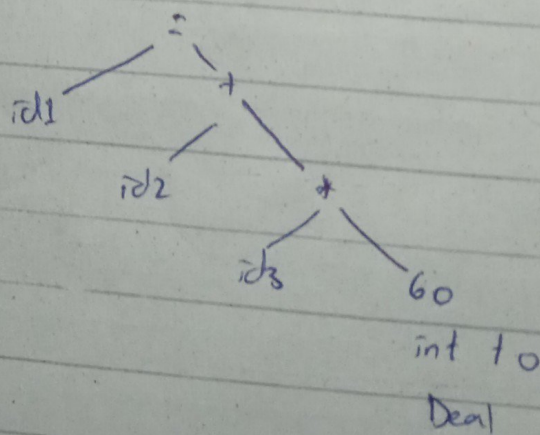
Lexical Analyzer



$id1 = id2 + id3 * 60$



Semantic Analysis



• Adding further information
in syntax tree

a^+ or aa^+ same

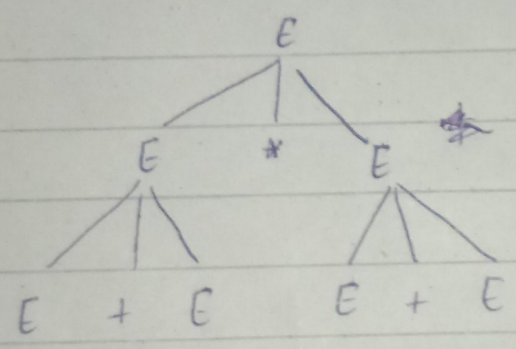
* Patterns are defined using Regular Expression.

* example letter (letter + digit)*

$$S \rightarrow aSa$$

$\hookrightarrow$ sentence of a language

$$a + b * c + a$$



$\rightarrow$ linear Form

Left Most derivation

$E \rightarrow E + E$
 $\quad id + E$
 $\quad id + E * E$
 $\quad id + id * E$
 $\quad id + id * E + E$
 $\quad id + id * id + E$
 $\quad id + id * id + id$
 $\quad a + b * c + a$

Right Most derivation

$E \rightarrow E + E$
 $\quad E + id$
 $\quad E * E + id$
 $\quad E * id + id$
 $\quad E + E * id + id$
 $\quad E + id * id + id$
 $\quad id + id * id + id$
 $\quad a + b * c + a$

- If any Grammar has more than one LMD or RMD then the grammar is Ambiguous and not acceptable.
- A good grammar is not Ambiguous.

Exercise abab

$S \rightarrow aSbS \mid bSaS \mid \epsilon$ is ambiguous b/c of two derivation tree

~~S~~ $\Rightarrow$

